

Introduction à Spark

Module 1 — L'écosystème Big Data et Apache Spark / PySpark

Michel Cadennes — avril 2026

Objectifs pédagogiques

- Expliquer les limites des traitements classiques face au Big Data
- Décrire l'architecture interne de Spark et le rôle de ses composants principaux
- Distinguer Spark de Hadoop/MapReduce et justifier les gains de performance
- Identifier les bibliothèques de l'écosystème Spark et leurs cas d'usage
- Comprendre le positionnement de PySpark comme interface Python de Spark

1. Contexte : les Big Data

L'explosion des données

Depuis 2010, une croissance exponentielle des volumes produits à l'échelle mondiale

Volume

Téraoctets à pétaoctets de données

Vélocité

Données produites et traitées en quasi-temps réel

Variété

Structuré (SQL), semi-structuré (JSON), non structuré (texte, image)

Véracité

Qualité et fiabilité des données incertaines

Valeur

La donnée brute doit être transformée pour créer de la valeur

Pourquoi les outils classiques ne suffisent plus

Traiter 10 To de logs sur une seule machine avec Pandas : problèmes immédiats

⚠ Mémoire insuffisante

Pandas charge tout le jeu de données en RAM — impossible au-delà de quelques Go par machine

🕒 Temps prohibitif

Un seul processeur ne peut pas paralléliser le travail — le traitement séquentiel ne passe pas à l'échelle

✗ Pas de tolérance aux pannes

Si la machine tombe pendant le calcul, tout le travail est perdu et doit recommencer depuis zéro

Solution : distribuer le traitement sur un **cluster de plusieurs machines** — c'est l'objectif des frameworks de calcul distribué comme Apache Spark.

2. Histoire et philosophie de Spark

De Hadoop à Spark

Spark est né en 2009 à l'AMPLab (Berkeley) pour résoudre le problème des I/O de Hadoop

Hadoop MapReduce (2006)

- HDFS : stockage distribué tolérant aux pannes
- MapReduce : Map local + Reduce global
- Chaque étape = lecture + écriture sur disque
- Très lent pour les pipelines itératifs (ML)

10–100x

plus rapide

Apache Spark (2009 → Apache 2013)

- Données intermédiaires conservées en RAM
- Lazy evaluation + DAG optimisé
- API unifiée : SQL, ML, Graphes, Streaming
- APIs Python, Scala, Java, R

Hadoop : Disque → Map → Disque → Map → Disque → Reduce → Disque

Spark : Disque → [Map → Map → Reduce] → Disque (tout en RAM)

Les principes philosophiques de Spark

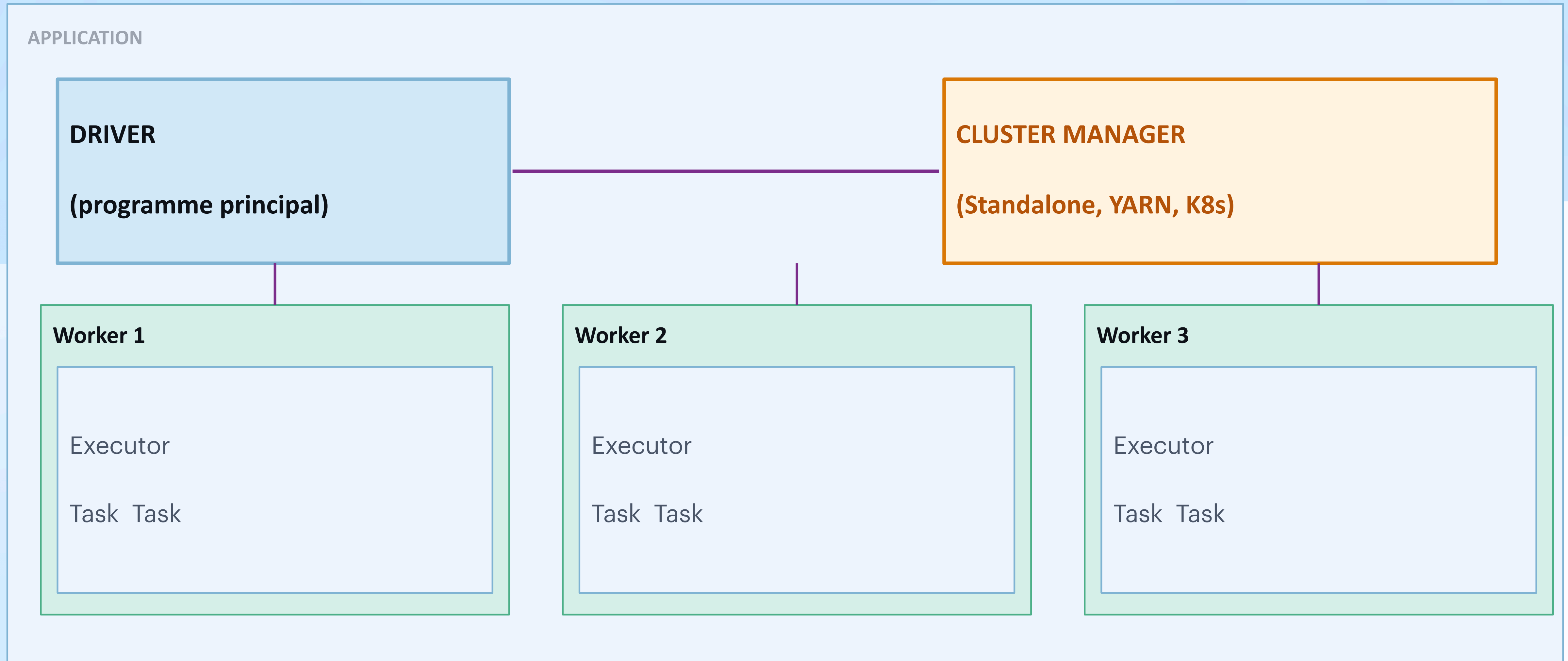
Principe	Description
Calcul en mémoire	Les données intermédiaires sont stockées en RAM, pas sur disque
Lazy evaluation	Les transformations ne s'exécutent pas immédiatement — elles sont planifiées dans un DAG
Tolérance aux pannes	Les données perdues peuvent être recalculées grâce au lignage (lineage)
API unifiée	Un seul framework pour le batch, le streaming, le SQL, le ML et les graphes
Polyglotte	APIs officielles en Scala, Java, Python (PySpark) et R

Performance : 10× plus rapide que Hadoop sur disque · **100× plus rapide en mémoire** (*benchmarks AMPLab, Berkeley*)

3. L'architecture de Spark

Vue d'ensemble du cluster Spark

Trois types d'entités : Driver, Cluster Manager et Executors (sur les Workers)



Driver et Cluster Manager

Le Driver

- Contient la fonction `main()` de l'application
- Crée la `SparkSession` (point d'entrée unique)
- Construit le plan d'exécution (DAG)
- Coordonne les Executors et collecte les résultats

⚠ **Point clé :** le Driver tourne sur une seule machine. `collect()` peut saturer sa mémoire si le volume est trop important.

Le Cluster Manager

- Standalone — cluster Spark dédié, simple
- YARN — environnements Hadoop existants
- Kubernetes — déploiements cloud-native
- Local — développement & tests (`local[*]`)

Rôle : gérer les ressources (CPU, RAM) du cluster et allouer les Executors sur les nœuds Workers.

Les Executors — Le moteur du calcul

Les processus qui s'exécutent sur les nœuds workers et effectuent réellement les calculs

- Reçoivent des tâches (tasks) depuis le Driver
- Stockent les données en mémoire (cache) ou sur disque
- Renvoient les résultats au Driver

Job déclenché par une action (count(), collect(), save()...)

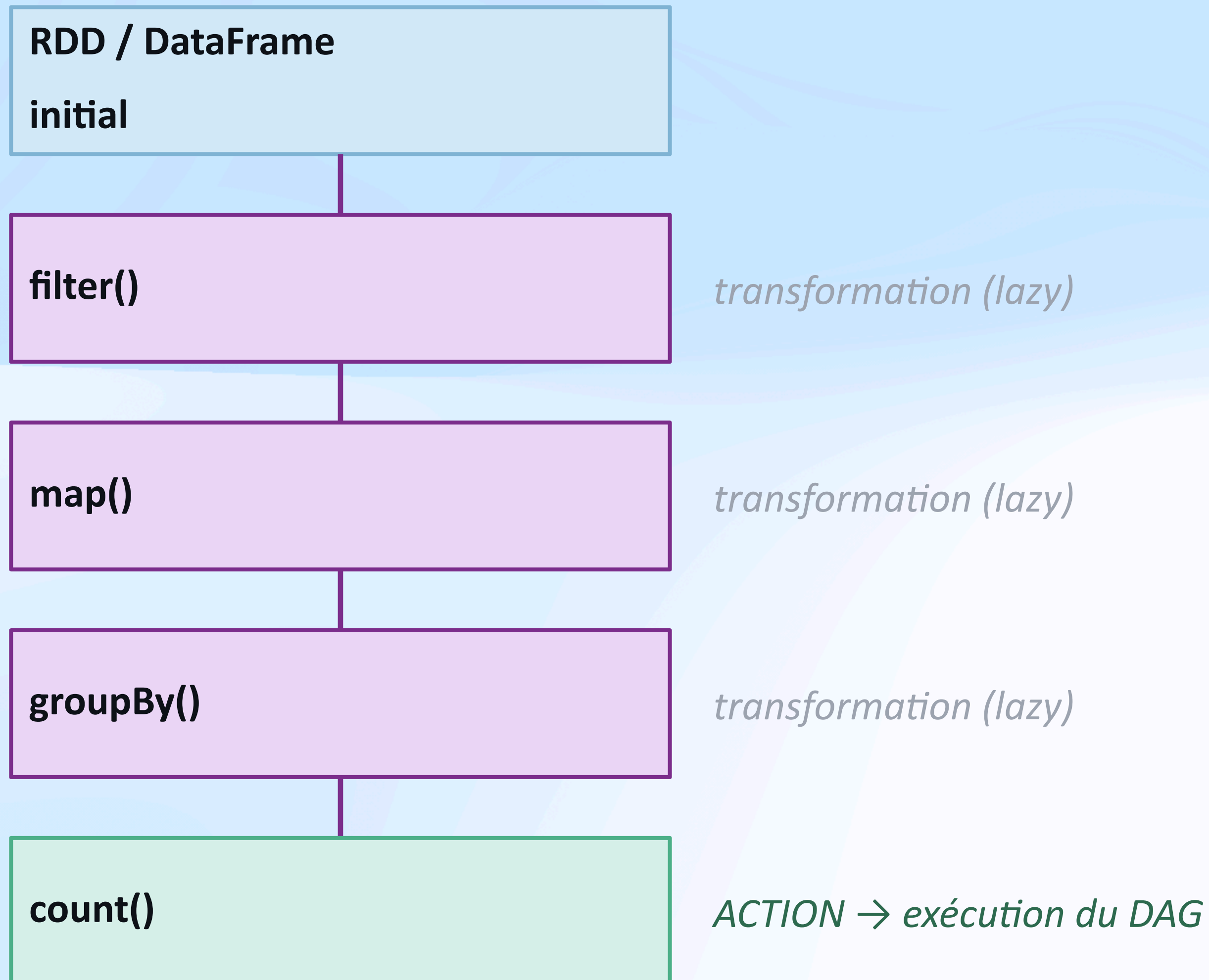
└ **Stage** ensemble de tâches sans shuffle réseau entre elles

└ **Task** unité de travail minimale — s'applique à une partition de données

→ 1 partition de données = 1 task = 1 thread sur un Executor

Le DAG — Directed Acyclic Graph

Spark construit un plan d'exécution optimisé avant de lancer le moindre calcul (lazy evaluation)



Optimisation automatique

Spark réordonne les opérations, fusionne les filtres, élimine les calculs inutiles avant l'exécution

Recalcul en cas de panne

Le lignage (lineage) du DAG permet de recalculer uniquement les partitions perdues sans tout recommencer

4. L'écosystème de Spark

Spark : une plateforme unifiée

Bien plus qu'un moteur de calcul — un écosystème de bibliothèques spécialisées

SPARK CORE — RDD, scheduling, mémoire, I/O, shuffle, tolérance aux pannes

Spark SQL

DataFrames & Datasets

- SQL standard
- Catalyst Optimizer
- Lecture CSV/JSON/Parquet

MLlib

Machine Learning distribué

- Régression/
Classification
- Clustering, ALS
- Pipelines ML

GraphX / GraphFrames

Traitement de graphes

- PageRank
- Détection de communautés
- Composantes connexes

Structured Streaming

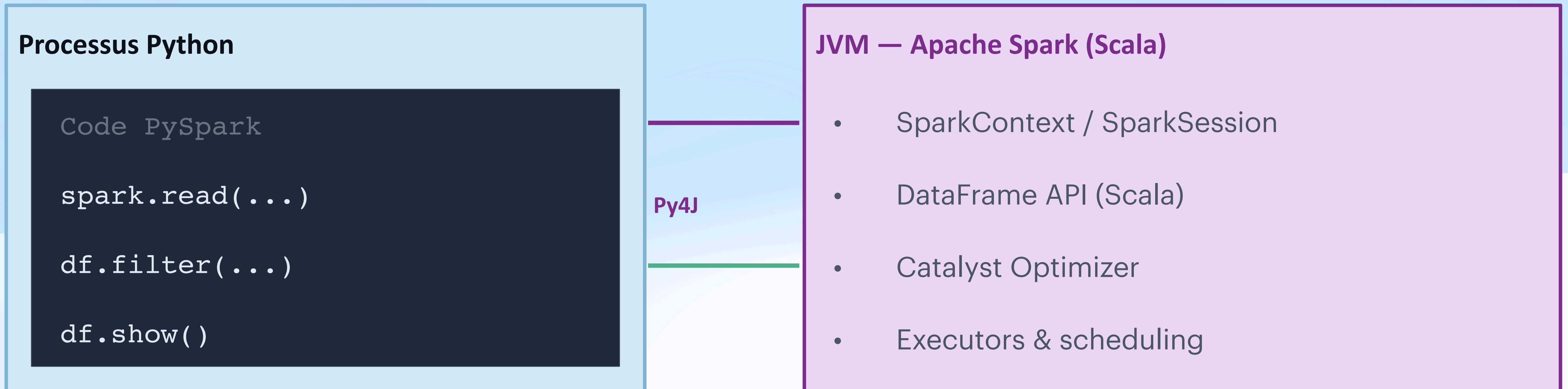
Flux en temps réel

- Kafka, sockets
- Même API que batch
- Micro-batch ou continu

5. PySpark = Python + Spark

Architecture PySpark — le pont Py4J

PySpark repose sur Py4J : un pont entre l'interpréteur Python et la JVM où tourne Spark



- Performance identique à Scala pour les DataFrames (plan d'exécution produit côté JVM)
- Pour les UDF Python pures : légère surcharge due à la sérialisation Python ↔ JVM
- Pandas UDFs (PyArrow) : fonctions Python vectorisées exécutées directement sur les Executors

Pourquoi PySpark pour l'IA ?

Écosystème Python

Compatibilité native avec NumPy, Pandas, scikit-learn, PyTorch, TensorFlow

Pandas API on Spark

`pyspark.pandas` — API quasi-identique à Pandas, mais exécutée sur le cluster

Pandas UDFs

Fonctions Python vectorisées via PyArrow — performances proches du Scala natif

MLlib + Python

Accès à tous les algorithmes ML distribués (ALS, GBT, PageRank...) depuis Python

Notebooks

Intégration native avec Jupyter, Databricks, Google Colab, VS Code

Premier programme PySpark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MonPremierProgramme") \
    .master(
"local[*]") \
    .getOrCreate()

df = spark.read.option("header", True).csv("data/utilisateurs.csv")
df.filter(df["age"] > 25).select("nom", "age").show()
spark.stop()
```

- **local[*]** : tous les cœurs disponibles de la machine (mode développement)
- **SparkSession** : point d'entrée unique depuis Spark 2.x — remplace SparkContext et SQLContext
- **Spark UI** disponible sur <http://localhost:4040> pendant l'exécution

Spark vs les alternatives

Critère	Spark	Pandas	Dask	Ray
Scalabilité	Très haute (PB)	Faible (RAM machine)	Haute	Haute
Calcul distribué	Natif	Non	Oui	Oui
SQL	Spark SQL natif	Limité	Limité	Non
ML distribué	MLlib	scikit-learn	Non natif	Ray AIR
Streaming	Structured Streaming	Non	Limité	Non
Courbe apprentissage	Élevée	Faible	Moyenne	Moyenne
Cas d'usage idéal	Big Data, ETL, ML @ scale	Prototypage	Données > RAM	ML/RL distribué

Résumé du module

Concept	Points clés à retenir
Big Data	Volume, Vitesse, Variété — les outils classiques (Pandas, 1 machine) ne suffisent plus
Spark vs Hadoop	Spark garde les données intermédiaires en RAM → 10 à 100× plus rapide
Driver	Chef d'orchestre — construit le DAG, coordonne, mais ne calcule pas
Executors	Calculent réellement les données, en parallèle sur les Workers
Lazy evaluation	Les transformations sont planifiées dans le DAG, pas exécutées immédiatement
DAG	Plan d'exécution optimisé automatiquement, déclenché par une action
Écosystème	Spark SQL, MLlib, GraphX/GraphFrames, Structured Streaming
PySpark	Python + Py4J → accès complet à Spark, compatible avec tout l'écosystème IA Python

Exercices

Ex. 1 — Réflexion *(10 min)* Une entreprise analyse 5 To de logs. Pourquoi Pandas ne convient-il pas ? Quels composants Spark seraient impliqués ?

Ex. 2 — Schéma d'architecture *(15 min)* Dessinez un cluster Spark : 1 Driver, 1 Cluster Manager, 4 Workers (2 Executors chacun). Indiquez les flux lors d'un count().

Ex. 3 — Mise en pratique — Jupyter / Colab *(30 min)* Installer PySpark, créer une SparkSession locale, créer un DataFrame depuis une liste Python, filtrer une colonne, observer le DAG dans la Spark UI (port 4040).

Pour aller plus loin

Documentation & Ressources

- Documentation officielle Spark : spark.apache.org/docs/latest
- Databricks Academy — cours PySpark gratuits : academy.databricks.com
- Spark Summit talks : YouTube, chaîne Databricks

Article fondateur

Zaharia et al. — *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*

NSDI 2012 — le papier fondateur qui a posé les bases de l'architecture Spark / RDD

Module suivant

→ **Module 2** : Installation et prise en main — configuration de l'environnement Spark local, Docker, Databricks